Razonamiento Automático 2026/27
Challenge 4: Enunciado de entrenamiento

Tiro al plato: push/pop, ECS muy básica, multi-fichero, gráficos y animación

El juego: tiro al plato

Un plato (`clay`) vuela cruzando la pantalla. Tú eliges por dónde disparar y disparas: si el disparo cruza al plato, ***¡impacto!*** 💥. Lo vas a construir por partes: el motor (dado, en `engine.asm`) ya sabe leer el mando y dibujar dígitos; tú escribes la lógica del juego (`game.asm`): mover el plato, mover el disparo, comprobar el impacto, llevar la cuenta y animar.

De paso, empezarás a adentrarte en la arquitectura *Entidad-Componente-Sistema* (ECS). Esta arquitectura separa los datos (componentes) del código que los procesa (sistemas) y los organiza para que la CPU los recorra de forma sencilla y eficiente. Hoy en día es muy usada en videojuegos.

> ⓘ **Por qué "ECS muy básico"**
> No vas a usar arrays de entidades todavía (eso llega en el Challenge 5, de equipo). Aquí cada "entidad" (`clay` , `shot`) son simplemente unos pocos datos con nombre, y cada **sistema** es una rutina que actualiza una entidad cada frame (`system_move_clay` , `system_move_shot` ...). Es la misma idea de ECS (separar entidad y sistema), a la escala mínima que hace falta para no perdernos en abstracción antes de tocar código real.

Objetivos de aprendizaje

De repaso (ya entrenado, reutilizado en un contexto nuevo):

- Direcciones del tilemap (`fila*32+columna`), lectura del pad, subrutinas con `call` / `ret` , devolver resultados en el flag
- Trazar/depurar código ajeno con errores de sintaxis y de lógica

Nuevos en este Challenge:

- `push` / `pop` : conservar un valor en un registro cuando una llamada intermedia lo va a destruir.
- **De constante de diseño a parámetro**: cuándo merece la pena que un valor "fijo" (como una fila o un límite) llegue por parámetro en vez de estar escrito con `DEF` , para poder cambiarlo (p. ej. subir la dificultad) sin volver a ensamblar.
- **Organización en varios ficheros**: cómo usar símbolos definidos en otro fichero .
- **Gráficos propios**: definir tus propios tiles (patrones de píxeles, `DB` , formato 2bpp) y copiarlos a VRAM, con una rutina de copia **genérica y reutilizable** (`memcpy`); la misma idea de "de constante a parámetro", pero para una rutina entera.
- **Animación básica**: alternar el tile visible de una entidad para dar sensación de movimiento.
- **Negar un valor de 8 bits**: complemento a dos con `cpl` (invierte los bits) + `inc a` , igual que ya "sumaste el negativo" para restar de 16 bits sin `sub hl,rr` .
- **Elegir el flag de retorno**: cada rutina devuelve el sí/no en el flag más práctico para ella (`Z` para una igualdad, carry `C` para "menor que") y lo indica en su contrato.

Código común: `engine.asm`

> ⚠ **Asume que este fichero ya existe y funciona. Tus rutinas (en `game.asm`) lo usan tal cual; no hace falta entender cada línea, solo el contrato (entradas/salidas) de cada rutina.**

engine.asm

```
;; =====================
;; Constants (exported: visible from game.asm)
;; =====================
EXPORT DEF TILEMAP      = $9800
EXPORT DEF ROW_STRIDE   = $20

EXPORT DEF BLANK_TILE   = 0
EXPORT DEF CLAY_TILE_A  = 1
EXPORT DEF CLAY_TILE_B  = 2
EXPORT DEF SHOT_TILE    = 3
EXPORT DEF TILE_DIGIT_0 = 4  ;; tiles $04-$0D
                             ;; = digits 0-9 (given)
EXPORT DEF ROW_CLAY     = 5
EXPORT DEF CLAY_MIN_COL = 2
EXPORT DEF CLAY_MAX_COL = 17
EXPORT DEF CLAY_STEP    = 1

EXPORT DEF ROW_SHOT_START = 12
EXPORT DEF CURSOR_MIN_COL = 2
EXPORT DEF CURSOR_MAX_COL = 17

EXPORT DEF HIT_TOLERANCE  = 1

EXPORT DEF HUD_ROW           = 0
EXPORT DEF HUD_ATTEMPTS_COL  = 0
EXPORT DEF HUD_ATTEMPTS_CELL = TILEMAP + HUD_ROW*ROW_STRIDE + HUD_ATTEMPTS_COL

;; =====================
;; Data (WRAM)
;; =====================
SECTION "Clay", WRAM0
clay_col::    DS 1        ;; current column of the clay (0-31)

SECTION "Shot", WRAM0
shot_col::    DS 1
shot_row::    DS 1
shot_active:: DS 1        ;; 0 = no shot in flight, 1 = yes

SECTION "Cursor", WRAM0
cursor_col::  DS 1        ;; column the next shot will fire from

SECTION "HUD Vars", WRAM0
attempts::    DS 1        ;; shots resolved
hits::        DS 1        ;; hits

SECTION "Anim Vars", WRAM0
anim_frame::  DS 1        ;; 0 or 1: which tile to show

;; =====================
;; Engine Routines
;; =====================
SECTION "Engine Code", ROM0

```

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
wait_vblank::
   ;; (real implementation omitted here)
   ;; (waits for rLY=144, as in previous Challenges)
   ret

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; 📥 INPUT: B=row, C=column
;; 🔙 RETURNS: HL=address in the tilemap
;; ❌ May modify A, BC, DE
;; Same idea as board_cell_address (Challenge 3)
;; but for the tilemap (32 wide)
tile_address::
   ld  a, b
   ld  l, a
   ld  h, 0
   add hl, hl  ;; /
   add hl, hl  ;; | hl = row * 32
   add hl, hl  ;; |  5 doublings
   add hl, hl  ;; |  same as *8 with 3
   add hl, hl  ;; \  in Ex.3 of Challenge 3
   ld  de, TILEMAP
   add hl, de
   ld  a, c
   ld  e, a
   ld  d, 0
   add hl, de
   ret
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves cursor_col with Left/Right,
;; with limits (same as Ex.11-12 of Challenge 3)
;; ❌ Modifies A, B, HL
read_pad_and_move_cursor::
   ld  hl, $FF00
   ld  a, %00100000
   ld  [hl], a
   ld  a, [hl]
   ld  a, [hl]
   ld  a, [hl]
   ld  b, a
   ld  hl, cursor_col
   bit 1, b
   jr  nz, .no_left
   ld  a, [hl]
   cp  CURSOR_MIN_COL
   jr  z, .no_left
   dec [hl]
.no_left
   bit 0, b
   jr  nz, .no_right
   ld  a, [hl]
   cp  CURSOR_MAX_COL
   jr  z, .no_right
   inc [hl]
.no_right
   ret

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; 🔙 RETURNS (flag):
;;  Z if A is pressed right now, NZ if not
;; ❌ Modifies A, HL
```

```
check_fire_button::
   ld  hl, $FF00
   ld  a, %00010000
   ld  [hl], a
   ld  a, [hl]
   ld  a, [hl]
   ld  a, [hl]
   bit 0, a
   ret

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Table with Tiles for digits 0-9
DigitTileTable:
   DB TILE_DIGIT_0+0, TILE_DIGIT_0+1, TILE_DIGIT_0+2, TILE_DIGIT_0+3, TILE_DIGIT_0+4
   DB TILE_DIGIT_0+5, TILE_DIGIT_0+6, TILE_DIGIT_0+7, TILE_DIGIT_0+8, TILE_DIGIT_0+9

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; 📥 INPUT: A=digit (0-9)
;; 🔙 RETURNS: A=tile
;; ❌ Modifies DE, HL
digit_to_tile::
   ld  hl, DigitTileTable
   ld  e, a
   ld  d, 0
   add hl, de
   ld  a, [hl]
   ret
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
main::
   call game_loop  ;; game_loop:: you define it, in game.asm
   jr  main
```

ⓘ ¿Para qué sirven `EXPORT` y `::`?

Marcan un símbolo como ***global***: `EXPORT` para constantes (`DEF`) y `::` para etiquetas. El traductor (`rgbasm`) lo ***exporta***: lo deja visible en el fichero `.o` de salida. Así el ***linker*** puede encontrarlo cuando otro fichero `.o` lo usa.

Nota (GBTelera): su Makefile ensambla con este flag:

```
ASMFLAGS := -E -Weverything
```

`-E` significa "*export all labels*": las **etiquetas** son globales (¡ojo!, los `DEF` no, requieren `EXPORT`), así que `::` no hace falta mientras se use el flag.

Tu código: `game.asm`

Todo lo que sigue va en `game.asm`, un fichero **distinto** de `engine.asm`. Se ensamblan por separado y luego se enlazan los resultados, así:

```
$ rgbasm  engine.asm -o engine.o  # Assemble engine.asm
$ rgbasm  game.asm   -o game.o    # Assemble game.asm
$ rgblink engine.o game.o -o juego.gb # Link both into game ROM file
```

Este proceso, automatizado con GBTelera, incluye `-E` en cada `rgbasm`, ***exportando*** todas las etiquetas a `engine.o` y `game.o` como globales. Así, `rgblink` (el ***linker***) las ve y puede resolver las referencias entre `engine.o` y `game.o`.

Ejercicio 1

Implementa `system_move_clay`

El plato vuela por la fila `ROW_CLAY`. Cada llamada: borra el tile en su columna actual (`clay_col`), avanza la columna 1 (`CLAY_STEP`) y dibuja el tile `CLAY_TILE_A` en la nueva columna. Para este ejercicio no hace falta gestionar qué pasa al llegar a `CLAY_MAX_COL` (lo verás en el Ejercicio 5).

Usa `tile_address` (dada) para calcular la dirección: `B`=fila, `C`=columna.

system_move_clay

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; "System" that moves the clay one step,
;; clearing the old cell and drawing the new one.
;;
system_move_clay::
    ;;; <<YOUR CODE>>
```

Ejercicio 2

Completa `system_move_shot`

`system_move_shot` mueve el disparo una fila hacia arriba cada llamada, y cuando llega a `ROW_CLAY` resuelve el disparo (comprueba el impacto y lo desactiva). Completa los huecos.

Asume que `system_check_hit` (Ejercicio 3) ya existe y funciona: devuelve el resultado en el flag carry (`C`=impacto, `NC`=fallo).

Cada hueco `;_____` debe ser completado con una instrucción. Los comentarios no siempre dicen *literalmente* cuál es: algunos sí, otros solo explican y hay que razonarlo.

system_move_shot (completa el código)

```
;; Moves the shot one row upward each call.
;; When it reaches ROW_CLAY, resolves the shot: checks
;; for a hit and deactivates it either way.
system_move_shot::
   ld  a, [shot_active] ;; A = indicates if a shot is active
   or  a                ;; Is A == 0? (no shot is active)
   ;__________          ;; If no shot is active, return
                        ;;   (nothing to do)

   ;; Erase the shot at its current cell
   ld  a, [shot_row]  ;; A = the shot's current row
   ld  b, a           ;; B = row, for tile_address's contract
   ld  a, [shot_col]  ;; A = the shot's current column
   ld  c, a           ;; C = column, for tile_address's contract
   call tile_address  ;; HL = address of the shot's current cell
   ;__________        ;; Clear the cell

   ;; Move the shot ONE ROW closer to the clay (upwards)
   ld  a, [shot_row]  ;; /
   ;__________        ;; | Move shot_row 1 ROW Upwards
   ld  [shot_row], a  ;; \

   ;; Check if it reaches the CLAY ROW
   cp  ROW_CLAY       ;; has the shot reached the clay's row?
   ;__________        ;; if it hasn't arrived yet, .keep_flying

.clay_row_reached
   ;; The shot reached the clay's row: find out whether it
   ;; was a hit, but only count it in `hits` when it was a hit
   ;__________        ;; Call the routine that checks for a hit
   ;__________        ;; It was a miss (NC): skip past the count
   ld  hl, hits       ;; HL = address of the hits counter
   ;__________        ;; hits counter += 1
.no_hit
   ld  hl, attempts  ;; HL = address of the attempts counter
   inc [hl]          ;; every resolved shot counts as an attempt
   xor a               ;; A = 0
   ld  [shot_active], a ;; the shot is resolved either way
   ret

.keep_flying
   ;; Draw the shot at its new row
   ld  b, a           ;; B = new row (A still holds the new row)
   ld  a, [shot_col]  ;; / C = shot's column
   ld  c, a           ;; \   (for tile_address's contract)
   call tile_address  ;; HL = address of the shot's new cell
   ;__________        ;; Draw the shot
   ret
```

Ejercicio 3

Reescribe `system_check_hit`

La versión de abajo funciona (comprueba si `shot_col` está a `HIT_TOLERANCE` columnas o menos de `clay_col`), pero hace el doble de restas de las necesarias, y usa `hl` sin necesitarlo. Reescríbela.

Restricciones clave

- Calcula la diferencia con una única resta (no dos comparaciones de rango por separado)
- No puedes usar `hl`
- Mantén el contrato: resultado en el flag carry (`C` =impacto, `NC` =fallo)

ⓘ ¿Por qué en el carry?
`cp X` activa el carry (`C`) cuando `A < X` . Así, una comprobación de "menor o igual" deja el resultado directamente en el carry, sin tener que forzar `Z` / `NZ` . Cada rutina devuelve el sí/no en el flag que le resulta más práctico, y lo indica en su contrato.

ⓘ Cómo calcular el *opuesto* en 8 bits
En aritmética modular `-A` (el opuesto de `A`) es el ***complemento a 2*** de `A` : se calcula en 2 pasos:

1. Invirtiendo todos los bits de `A` (`cpl a`)
2. Sumando 1 al resultado (`inc a`)

Así, si una resta te da un resultado negativo, puedes obtener su valor positivo (su opuesto).

Código a reescribir

```
;; 📥 shot_col, clay_col in WRAM. 🔙 Carry: C=hit, NC=miss
system_check_hit::
   ld  a, [clay_col]
   ld  hl, HIT_TOLERANCE
   sub l
   ld  b, a                  ;; b = lower bound

   ld  a, [clay_col]
   add a, l
   inc a
   ld  c, a                  ;; c = upper bound + 1

   ld  a, [shot_col]
   cp  b
   jr  c, .miss              ;; shot_col < lower bound
   cp  c                     ;; carry if shot_col < upper bound + 1: hit
   ret
.miss
   or  a                     ;; or always clears the carry: NC = miss
   ret
```

Ejercicio 4

? Implementa `system_draw_hud`
Escribe `attempts` (0-9) como dígito en la celda `HUD_ATTEMPTS_CELL` del HUD.

ϟ Técnica clave
`digit_to_tile` (asume que ya existe) recibe un dígito en `A` (0-9) y devuelve su ID de tile en `A` para poder dibujarlo.
¡Pero ojo! ❌ modifica `HL` . Necesitas `HL` **antes** de llamarla (para saber dónde escribir) y **después** (para escribir ahí): consérvalo con `push` / `pop` .

system_draw_hud

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Draws `attempts` as a digit on the HUD.
;;
system_draw_hud::
    ;;; <<YOUR CODE>>
```

Ejercicio 5

De constante a parámetro

¿Y si queremos que el plato vuele a distintas alturas? ¿Que tal si cada vez sale una fila más arriba, incrementando la dificultad?

Reescribe `system_move_clay` para que la fila (hasta ahora la constante `ROW_CLAY`) le llegue como parámetro en el registro `A`.

Pista clave

Según su contrato, `tile_address` puede modificar `A`, `BC` y `DE`, y aquí la llamas **dos veces** (para borrar y para dibujar). Piensa cómo conservar la fila entre medias: ya conoces la técnica.

system_move_clay (row as a parameter, with wraparound)

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Like Exercise 1, but complete: the row now
;; arrives as a parameter (instead of ROW_CLAY),
;; and it wraps around when it passes CLAY_MAX_COL.
;; 📥 INPUT: A = row the clay flies on
;;
system_move_clay::
    ;;; <<YOUR CODE>>
```

Reflexión

¿Para qué sirve este cambio, más allá de "hacer lo mismo de otra forma"? ¿Qué habrías tenido que hacer sin él, para tener platos a distintas alturas en distintos niveles?

Ejercicio 6: Multi-fichero

6.1. Explica

Todas tus rutinas `, etc.` llevan doble `::`. Si `tile_address` estuviera declarada en `engine.asm` como `tile_address:` (un solo `:`), y la llamases desde `game.asm` con `call tile_address`, ensamblando **sin** `-E`, ¿en qué fase fallaría: al ensamblar `engine.asm`, al ensamblar `game.asm`, o al enlazar? ¿Por qué?

6.2. Explica

`TILEMAP` está declarada como `EXPORT DEF TILEMAP = $9800`. Si quitas `EXPORT` y la usas desde `game.asm`, ¿en qué fase falla?

Pista

`rgbasm` traduce cada fichero **por separado** a un `.o`; solo entonces `rgblink` los combina en un `.gb`. Una etiqueta o constante que "el otro fichero no ve" no es un problema de sintaxis dentro de su propio fichero.

Ejercicio 7: `memcpy`

Gráficos propios: `load_clay_tile`

Copia el patrón de píxeles del plato (`ClayTileData`, dado abajo) a la zona de datos de tiles en VRAM, en el tile `CLAY_TILE_A`. En vez de escribir el bucle de copia directamente dentro de `load_clay_tile`, escríbelo como una rutina aparte y **genérica**, `memcpy`: dado un origen, un destino y un número de bytes, copia sin saber nada de tiles ni de platos. Luego úsala desde `load_clay_tile`.

Formato de un tile (2bpp)

Un tile ocupa 16 bytes en la VRAM (8 filas × 2 bytes/fila). La zona de datos de tiles empieza en `$8000`; el tile `N` ocupa `$8000 + N*16`.

Dato clave

Igual que con el tilemap: la escritura debe suceder durante VBLANK, porque estamos escribiendo en VRAM. Impleméntala asumiendo que se ejecutará durante VBLANK.

memcpy / load_clay_tile (completa los huecos)

```
ClayTileData:
   DB %00111100, %00111100
   DB %01000010, %01000010
   DB %10000001, %10000001
   DB %10000001, %10000001
   DB %10000001, %10000001
   DB %01000010, %01000010
   DB %00111100, %00111100
   DB %00000000, %00000000

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; 📥 HL = source, DE = destination, B = number of bytes
;; Copies B bytes from [HL] onwards to [DE] onwards.
;; Reusable: works for tiles, or for copying ANY block of bytes.
;;
memcpy::
.loop
   ;__________    ;; read one byte, HL += 1
   ;__________    ;; write the byte to [DE]
   ;__________    ;; DE += 1. Both HL & DE point to next bytes
   dec b          ;; /
   jr  nz, .loop ;; \ Loop until counter (B) reaches 0
   ret

;; Copies ClayTileData to tile CLAY_TILE_A, using memcpy
load_clay_tile::
   ld  hl, ClayTileData ;; HL = source: the tile data itself
   ;__________        ;; DE = destination:tile CLAY_TILE_A in VRAM
   ;__________          ;; B = bytes to copy (a tile is 16 bytes)
   call memcpy
   ret
```

Reflexión

`memcpy` no sabe nada de platos, tiles ni VRAM: solo copia bytes. ¿Para qué más, en este juego o en cualquier otro, podrías reutilizarla?

Ejercicio 8

Animación: `system_animate_clay`

Cada vez que se llama a `system_animate_clay`, debe alternar el tile del plato entre `CLAY_TILE_A` y `CLAY_TILE_B` (sin mover su columna).

Usa `anim_frame` (0 ó 1: qué tile toca). Invierte `anim_frame` (`xor 1`) y redibuja el plato en su columna actual con el tile que toque.

Sobre la cadencia

En un juego real, esto no se llamaría en cada frame (iría demasiado rápido): quien orquesta el juego decide cada cuántos frames llamarla. Aquí solo escribes el cambio de tile en sí.

system_animate_clay

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Alternates the clay's visible tile between
;; CLAY_TILE_A/B every time it's called.
;;
system_animate_clay::
    ;;; <<YOUR CODE>>
```